

Change request log

1. Concept Location

Step #	Description	Rationale
1	We first ran the Mango Source code.	We used Visual code IDE to run the code. Used the search tool in IDE after performing ant fullDeploy and ant reload commands
2	We logged in using our credentials and landed in the home page.	After logging in we were able to notice that the mic button was unmute by default. We hovered on the button to get keywords, that will help us to figure out the places that we have to make changes.
3	We searched for “mute” using the regular expression feature of the Visual Studio IDE’s search tool.	“mute” was the keyword that we were able to see when we hovered on the button. We got a lot of results and most of them were in the MiscDwr.java file
4	We inspected the “MiscDwr.java” and marked it as located	While we inspected the methods within the class, we were able to find that there were plenty of methods like toggleSilence, silenceAll, toggleUserMuted etc...) that might be of use to make the second change.
5	We also found some search results for the keyword mute in common.js and header.js files.	We inspected and thought that these files might be of some use when explored further so we marked these also as located
6	We then inspected the file User.java and marked it as located	While examining this class we found a transient variable named “muted”. We marked this as located, as this might be of use in further analysis.

Time spent (in minutes): 90 minutes

Classes and methods inspected:

- mangoSource\build\resources\common.js
- \mangoSource\build\resources\header.js
- \mangoSource\src\com\serotonin\mango\web\dwr\MiscDwr.java
 - toggleSilence
 - silenceAll
 - toggleUserMuted
- mangoSource\src\com\serotonin\mango\vo\User.java

2. Impact Analysis

Step #	Description	Rationale
1	We inspected the the class “MiscDwr” and some methods that we thought might be useful on analysis.	We examined the following methods: 1)toggleSilenceThis method Toggles the silence status of a specific event and it has nothing to do with muting the sensoers so we marked it as”not to be changed”. 2) toggleUserMuted-Toggles the muted status of the current user.Though this worked with the muted status we marked it as “to be changed” as we wanted to know what kind of effect will it have if we changed this method
2	In the impact analysis of common.js we came across the “setUserMuted” function and marked it as “to be changed”	This function is responsible for controlling the mute state of a user and updating the user interface accordingly. So we thought changing this method will have a positive impact towards the second change.
3	We then reviewed the header.js file and the “SoundPlayer” function and marked it as to be changed	We understood that the soundplayer was a constructor that represents a sound player object with methods to play, stop, mute, and repeat audio sounds. It utilizes the SoundManager library for sound management, allowing control over sound playback, mute state, and repeated playback with delays. We were unsure whether it had to be changed or not. But, then considering the functionalit we decided to make changes and later decide its impact.
4	Then we examined the User.java file	We inspected the methods in this class and found that they had nothing to do with the second change. Except for the transient Boolean variable “muted” nothing in this file proved to provide a positive impact

Time spent (in minutes): 45

- mangoSource\build\resources\common.js
 - setUserMuted
- \mangoSource\build\resources\header.js
 - SoundPlayer
- \mangoSource\src\com\serotonin\mango\web\dwr\MiscDwr.java
 - toggleSilence
 - toggleUserMuted
- mangoSource\src\com\serotonin\mango\vo\User.java
 - private boolean transient muted variable

3. Prefactoring (optional)

Step #	Description	Rationale
1	Updated the toggleUserMuted method in MiscDwr.java to handle cases where the mute status is initially undefined.	Ensured robustness by checking for null values before toggling the mute state. This modification prevents potential NullPointerExceptions and guarantees consistent behavior, improving the reliability of the application.

2	Modified the setUserMuted function in common.js to accept a default value for the muted parameter.	Enhanced usability and flexibility by providing a default value for the muted parameter. This simplifies function calls where the parameter is omitted, making the code more intuitive and reducing the likelihood of errors.
3	Reviewed and adjusted the User class in User.java to ensure proper serialization of the muted field.	Verified that the modification of the muted variable to a field does not interfere with the serialization process. This ensures that user preferences related to muting notifications persist across sessions as intended, maintaining data integrity and consistency.
4	Reviewed and ensured compatibility of the SoundPlayer class in header.js with other code modifications.	Confirmed that changes made elsewhere in the codebase (e.g., MiscDwr.java, common.js) do not adversely affect the functionality of the SoundPlayer class. This ensures that sound-related functionality remains consistent and stable across the application.

Time spent (in minutes): 20 minutes

4. Actualization

Step #	Description	Rationale
1	In order to disable the volume button by default, we changed public boolean toggleUserMuted() in MiscDwr.java	The code modification introduces a null check for the isMuted() method, ensuring proper handling when the user's muted state is not explicitly set. Additionally, it replaces the primitive boolean type with a Boolean wrapper class for the isMuted variable, enabling null values and providing flexibility for handling states of true, false, or null. Within the if-else block, the code toggles the muted state based on the current value, setting it to true if null, and returns the updated state after toggling. The steps involve identifying the need, introducing a null check, using the Boolean wrapper class, and ensuring the method returns the updated state

2	We then changed the setUserMuted method in common.js file	The change involves adding a default parameter muted = true to the setUserMuted function signature, allowing the function to be called without explicitly specifying a value for muted. This simplifies function calls by providing a default behavior of muting when no argument is given. The modification includes adding = true after the muted parameter. It ensures that the function logic remains correct with the default parameter value (true) and requires testing to ensure proper behavior in various scenarios, whether muted is explicitly provided or defaults to true.
3	We noticed that the SoundPlayer function in the header.js file also had been marked to be changed. So, we inspected it	We noticed that the mute property in the initial code was set to false, indicating that the volume is not mute by default so we changed it to be true. This would enable to set the mic as mute by default.
4	Finally, we changed the User.java file and set the transient Boolean variable to true from false.	The change involves modifying the initial value of the muted variable from false to true, setting its default state to true. To implement this, locate the declaration of the muted variable and adjust the assigned value to true. Ensure the modification aligns with the desired program behavior, considering any dependencies or implications of the variable's state.

Time spent (in minutes): 75

- mangoSource\build\resources\common.js
 - setUserMuted
- \mangoSource\build\resources\header.js
 - SoundPlayer
- \mangoSource\src\com\serotonin\mango\web\dwr\MiscDwr.java
 - toggleSilence
 - toggleUserMuted
- mangoSource\src\com\serotonin\mango\vo\User.java
 - private boolean transient muted variable

5. Postfactoring (optional)

Step #	Description	Rationale
1	Removed unnecessary code changes	After analyzing the test results, we decided that only the first change (private transient boolean muted = true;) is necessary. Therefore, we rolled back the other three changes (setUserMuted , toggleUserMuted , and SoundPlayer modifications) as they were deemed unnecessary based on test results.

2	Ran unit tests for the modified code	Executed unit tests for the class/module affected by the rollback to ensure that the necessary change did not introduce issues and that the functionality remains intact.
3	Validated the changes made to the User class in User.java.	Confirmed that the modification of the muted variable to a transient field properly maintains user preferences across sessions while ensuring data integrity. This validation ensures that user preferences related to muting notifications persist across sessions as
4	Ran system and integration tests	Checked the broader system and integration tests to confirm that the overall system behavior remains unaffected after the rollback.
5	Verified expected behavior in the UI	Manually verified the expected behavior in the user interface (UI), specifically checking the mute button functionality, to ensure it aligns with the desired outcome mentioned in Figure 4.
6	Reverted unnecessary code changes	Used version control (Git) to revert and discard unnecessary changes, maintaining the codebase in a clean state.
7	Re-executed all tests to confirm stability	Ran all tests (unit, system, integration, acceptance) again to confirm that the rollback did not introduce new issues and that the system remains stable.
8	Documented the decision and actions taken	Updated documentation to reflect the decision to keep only the necessary change and the rationale behind the rollback. This includes updating commit messages for clarity.

Time spent (in minutes): 70 minutes

6. Validation

Step #	Description	Rationale	Result
1	Unit testing: In each code file (i.e, in User.java, Misdwr.java, common.js, header.js) Set muted to true and check functionality	Confirm that setting muted to false works correctly	Pass {We, in fact observed that 3 changes are redundant and the only required change needed is in User.java}

2	Further Unit testing: Comment out the code changes in setUserMuted	Isolate the setUserMuted function from other changes	Pass
3	Comment out the code changes in toggleUserMuted	Isolate the toggleUserMuted function from other changes	Pass
4	Comment out the code changes in SoundPlayer	Isolate the SoundPlayer constructor from other changes	Pass
5	Set muted to false and check functionality in User.java	Isolate the User.java from other changes	Fail {Hence, we concluded that this is the required standalone change and stayed with this change alone rolling back others)
6	Regression Testing: Confirm default behavior without code changes	Verify that the notification sound is muted by default	Pass
7	Test notification sound toggle functionality	Manually toggle the notification sound and check behavior	Pass
8	Comment out the code changes in other redundant places	Verify that SoundPlayer is instantiated correctly	Pass
9	System Testing: Perform a system-level test with only 1st change	Ensure the system functions as expected with the first change	Pass
10	Integration Testing : Verify interaction with external modules	Ensure interactions with external modules are unaffected	Pass
11	Acceptance Testing: Verify the expected behavior as in Figure shown in assessment	Confirm that the notification sound is muted by default	Pass

Time spent (in minutes): 30 minutes

7. Summary of the change request

Phase	Time (minutes)	No. of classes inspected	No. of classes changed	No. of methods inspected	No. of methods changes
Concept location	90	4	4	3	1
Impact Analysis	45	4	4	4	4
Prefactoring	20	4	4	4	4
Actualization	75	4	4	4	4
Postfactoring	70	4	3	4	3
Verification	30	4	4	4	4
Total	330	4	4	4	20

8. Conclusions

The change request encountered challenges in handling user preferences for muting notifications, requiring intricate serialization and deserialization. Ensuring compatibility and integration across different system components posed difficulties, demanding meticulous attention and thorough testing. Validation, especially for exceptional cases, proved

challenging, necessitating comprehensive test cases. Time management was crucial, requiring careful planning to balance various phases of the change process. Despite challenges, effective collaboration, attention to detail, and perseverance led to successful implementation, providing valuable experience in managing complex codebase changes.